P1 2.5
P2 2.5
P3 2.0
P4 2.5
10.0

Nguyễn Phạm Thành Trung
N25DECE072
E25CQCE02-N

2.5 Problem 1: Singly Linked list - in-place reversal & pointer tracing

Given a singly linked list: head → [10] → [25] → [40] → [15] → Null

- Task 1: Write a function/algorithm in pseudocode to reverse the singly linked list including three pointer (pre, curr, next) to complete your function
- Task 2: Draw a diagram or step-by-step pointer tracer table showing the positions and values of the three pointer (prev, curr, next) after each iteration until the list is fully reversed

2.5 Problem 2: circular linked list - insert at head

Given a singly circular: head → [10] → [20] → [30] → points back to head

futurebook

Scanned with CamScanner

• Task 1: Write pseudocode for the function InsertHead(head, newValue) to insert a new node containing newValue at front of the circular linked list while maintaining its circular structure

• Task 2: Trace the execution steps (via text description or pointer diagram) when inserting newValue = 5 into the given list

2.5 Problem 3: Doubly link list - Insert before target & Edge case

Given a doubly linked list containing integers

• Task 1: Write pseudo code for Insert before(head, target, newValue) to insert a new node containing newValue before the first occure of Target

• Task 2: List the exact order of pointer assignment required and briefly just why this specific sequence avoid memory leaks or broken link

futurebook

Scanned with CamScanner

• Task 3: Explicitly handle...

1. Target is located at the head node
2. target does not exist in list

Problem 1

1.05 • Task 1:

```
Algorithm ReverseList(head):
    prev = Null
    curr = head
    next = Null
    While (curr != Null):
→       next = curr.next
→       curr.next = prev
→       prev = curr
→       curr = next
    head = prev
    return head
```

1.25 Task 2:

initial State: prev = Null, curr = Node(1
next = Null

futurebook

Scanned with CamScanner

Iteration	curr	next	Pointer	new prev	new curr	current list structure
initial	Node(10)	–	–	Null	Node(10)	10 → 25 → 40 → 15 → Null
iter 1	Node(25)	Node(40)	Node(10).next = Null	Node(10)	Node(25)	10 → Null
iter 2	Node(25)	Node(40)	Node(25). next = Node(10)	Node(25)	Node(40)	25 → 10 → Null
iter 3	Node(40)	Node(15)	Node(40). next = Node 25	Node(40)	Node(15)	40 → 25 → 10 → Null
iter 4	Node(15)	Null	Node(15). next = Node(40)	Node(15)	Null	15 → 40 → 25 → 10 → Null
final	Null	–	–	Node(15)	Null	set head = prev (Node 15)

→ Head → [15] → [40] → [25] → [10] → Null

Nguyễn Phạm Thành Trung
N25DEIE072
E25CQCE02-N

Problem 2:

Task 1

```
Algorithm InsertHead(head, newValue):
    NewNode = CreateNode(newValue)
    // Edge case: Empty list
    IF head == Nill:
        newNode.next = newNode
        head = newNode
        return head
    // Traverse to locate the tail node
    tail = head
    While tail.next != head:
        tail = tail.next
    //Insert new node at the Front
    newNode.next = head
    Tail.next = newNode
    head = newNode
    return head
```

futurebook

Scanned with CamScanner

1.6 Task 2:

Allocate new Node = 5

- Start at Tail = head (Node 10)
- Tail.next is Node(10) (≠ head)
  → update Tail = Node(20)
- Tail.next is Node(30) (≠ head)
  → update Tail = Node(30)
- Tail.next is Node(10) (== head)
  → Stop loop, Tail = Node(30) Found
- Set new Node.next = head
  → Node(5).next points to Node(10)
- Set tail.next = new Node
  → Node(30).next point to Node(5)
- Set head = new Node
  → head now points to Node(5)

Final Structure:

head → [5] → [10] → [20] → [30] ↩
(points back to [5])

futurebook

Scanned with CamScanner

25 Problem 3:

Task 1 and Task 3

```
Algorithm InsertBefore(head, target, newValue)
    if head == Null:
        return head

    if head.data == target:
        newNode = CreateNode(newValue)
        newNode.next = head
        newNode.prev = Null
        head.prev = newNode
        head = newNode
        return head

    curr = head
    while (curr != Null and curr.data
!= target):
        curr = curr.next

    if curr == Null:
        return head
```

futurebook

Scanned with CamScanner

```
newNode = CreateNode(newValue)
newNode.prev = curr.prev
newNode.next = curr
curr.prev.next = newNode
curr.prev = newNode

return head
```

0,75

• Task 2:

1. NewNode.prev = curr.prev
2. newNode.next = curr
3. curr.prev.next = newNode
4. curr.prev = newNode

futurebook

Scanned with CamScanner

Nguyễn Phạm Thạch Trung
N25 DECE072
E25 CQCE02-N

Problem 4

Task 1

```
Algorithm SortedInsert(head, val):
    newNode = CreateNode(val)

    IF(head == Null or val ≤ head.data):
        newNode.next = head
        head = newNode

    curr = head
    while(curr.next != Null and curr.next.data < val):
        curr = curr.next

    newNode.next = curr.next
    curr.next = newNode
    return head
```

futurebook

Scanned with CamScanner

1.2.5 • Task 2

Trace for inserting val = 6 into 2 → 5 → 8 :

Step	curr	curr.next	condition (curr.next != Null and curr.next.data < 6)	Action
Pre-check	–	–	head is not Null and 6 ≤ 2 is False	skip front insert
1	2	5	true and 5 < 6 is true	curr = 5
2	5	8	true and 8 < 6 is False	Stop
Link	5	8	–	node.next = 8, then 5.next = 6

⇒ result is: 2 → 5 → 6 → 8 → Null

futurebook

Scanned with CamScanner